



Documentation API - JobbingTrack

[← Retour au README principal](#)



Vue d'ensemble

JobbingTrack utilise une architecture de microservices moderne avec un **API Gateway central** qui route les requêtes vers **15+ services backend** spécialisés. Cette documentation couvre tous les endpoints disponibles, organisés par service.



Table des matières

-  [API Gateway](#)
 -  [Service d'authentification](#)
 -  [Service des candidatures](#)
 -  [Service des entreprises](#)
 -  [Service des contacts](#)
 -  [Service des entretiens](#)
 -  [Service des appels](#)
 -  [Service des événements](#)
 -  [Service de suivi](#)
 -  [Service des profils](#)
 -  [Service de notifications](#)
 -  [Service de workflows](#)
 -  [Service de dashboard](#)
 -  [Service de sécurité](#)
 -  [Service de métriques système](#)
 -  [Service de déploiement](#)
 -  [Service de statistiques Docker](#)
 -  [Codes d'erreur](#)
 -  [Authentification](#)
 -  [Exemples d'utilisation](#)
-



API Gateway

Base URL : `http://localhost:3000`

L'API Gateway est le point d'entrée unique pour tous les services. Il gère le routage, l'authentification centralisée, le rate limiting et la sécurité.

Endpoints de santé

Vérification de santé générale

GET /health

Réponse :

```
{
  "status": "healthy",
  "timestamp": "2025-01-27T10:00:00Z",
  "version": "4.0.0",
  "services": {
    "postgres": "healthy",
    "redis": "healthy",
    "auth-service": "healthy",
    "application-service": "healthy",
    "company-service": "healthy",
    "contact-service": "healthy",
    "interview-service": "healthy",
    "call-service": "healthy",
    "event-service": "healthy",
    "followup-service": "healthy",
    "profile-service": "healthy",
    "notification-service": "healthy",
    "workflow-service": "healthy",
    "dashboard-service": "healthy",
    "security-service": "healthy",
    "system-metrics-service": "healthy",
    "deployment-service": "healthy",
    "docker-stats-service": "healthy"
  },
  "uptime": "3600",
  "responseTime": "45ms"
}
```

Métriques Prometheus

GET /metrics

Réponse (format Prometheus) :

```
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",path="/health"} 1250
```

```
# HELP http_request_duration_seconds HTTP request duration in seconds
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{le="0.1"} 950
http_request_duration_seconds_bucket{le="0.5"} 1150
http_request_duration_seconds_bucket{le="1.0"} 1200
```

Middleware et sécurité

L'API Gateway applique automatiquement :

- **Authentification JWT** sur les endpoints protégés
- **Rate limiting** : 100 requêtes/15min par IP
- **CORS** : Configuration restrictive
- **Logging** : Traçabilité complète
- **Headers de sécurité** : HSTS, CSP, X-Frame-Options

Routage vers les services

Route	Service	Port	Description
/auth/*	Auth Service	3001	Authentification
/applications/*	Application Service	3002	Candidatures
/companies/*	Company Service	3003	Entreprises
/contacts/*	Contact Service	3004	Contacts
/interviews/*	Interview Service	3005	Entretiens
/calls/*	Call Service	3006	Appels
/events/*	Event Service	3007	Événements
/followups/*	Followup Service	3008	Suivi
/profiles/*	Profile Service	3009	Profils
/notifications/*	Notification Service	3010	Notifications
/workflows/*	Workflow Service	3011	Workflows
/dashboard/*	Dashboard Service	3007	Dashboard
/security/*	Security Service	3017	Sécurité
/system-metrics/*	System Metrics Service	3018	Métriques
/deployment/*	Deployment Service	3016	Déploiement

/docker-stats/*	Docker Stats Service	3015	Docker
-----------------	----------------------	------	--------

Service d'authentification

Base URL : `http://localhost:3000/auth` **Port direct :** `http://localhost:3001`

Authentification

Connexion

```
POST /auth/login
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "password123"
}
```

Réponse :

```
{
  "success": true,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "email": "user@example.com",
      "name": "John Doe",
      "role": "user",
      "permissions": ["read", "write"],
      "createdAt": "2025-01-27T10:00:00Z",
      "updatedAt": "2025-01-27T10:00:00Z"
    }
  },
  "timestamp": "2025-01-27T10:00:00Z"
}
```

Inscription

POST /auth/register

Content-Type: application/json

```
{
  "email": "newuser@example.com",
  "password": "password123",
  "name": "John Doe",
  "role": "user"
}
```

Réponse :

```
{
  "success": true,
  "data": {
    "user": {
      "id": "550e8400-e29b-41d4-a716-446655440001",
      "email": "newuser@example.com",
      "name": "John Doe",
      "role": "user",
      "status": "active",
      "createdAt": "2025-01-27T10:05:00Z"
    },
    "message": "Utilisateur créé avec succès. Un email de confirmation a été envoyé"
  }
}
```

Vérification du token

GET /auth/verify

Authorization: Bearer <access_token>

Réponse :

```
{
  "success": true,
  "data": {
    "valid": true,
    "user": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "email": "user@example.com",
      "role": "user",

```

```

    "permissions": ["read", "write"]
  },
  "expiresAt": "2025-01-27T11:00:00Z"
}

```

Rafraîchissement du token

```

POST /auth/refresh
Content-Type: application/json

{
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}

```

Réponse :

```

{
  "success": true,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "expiresAt": "2025-01-27T11:00:00Z"
  }
}

```

Déconnexion

```

POST /auth/logout
Authorization: Bearer <access_token>

```

Réponse :

```

{
  "success": true,
  "message": "Déconnexion réussie"
}

```

Réinitialisation du mot de passe

```
POST /auth/forgot-password
Content-Type: application/json
```

```
{
  "email": "user@example.com"
}
```

```
POST /auth/reset-password
Content-Type: application/json
```

```
{
  "token": "reset-token-here",
  "newPassword": "newpassword123"
}
```

Service des candidatures

Base URL : `http://localhost:3000/applications` **Port direct :** `http://localhost:3002`

Gestion des candidatures

Lister les candidatures

```
GET /applications
Authorization: Bearer <token>
```

Paramètres :

- `page` (optionnel) : Numéro de page (défaut: 1)
- `limit` (optionnel) : Nombre par page (défaut: 20)
- `status` (optionnel) : Filtrer par statut
- `companyId` (optionnel) : Filtrer par entreprise
- `sortBy` (optionnel) : Tri (date, status, company)

Réponse :

```
{
  "success": true,
  "data": {
```

```

"applications": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "userId": "user-id",
    "companyId": "company-id",
    "position": "Développeur Full Stack",
    "status": "applied",
    "description": "Mission passionnante...",
    "salaryMin": 45000,
    "salaryMax": 55000,
    "location": "Paris",
    "source": "LinkedIn",
    "notes": "Entreprise intéressante",
    "createdAt": "2025-01-27T10:00:00Z",
    "updatedAt": "2025-01-27T10:00:00Z",
    "company": {
      "id": "company-id",
      "name": "Tech Corp",
      "website": "https://techcorp.com"
    }
  }
],
"pagination": {
  "page": 1,
  "limit": 20,
  "total": 150,
  "totalPages": 8
}
}

```

Créer une candidature

POST /applications

Authorization: Bearer <token>

Content-Type: application/json

```

{
  "companyId": "550e8400-e29b-41d4-a716-446655440000",
  "position": "Développeur Full Stack",
  "status": "applied",
  "description": "Mission passionnante...",
  "salaryMin": 45000,
  "salaryMax": 55000,
  "location": "Paris",
  "source": "LinkedIn",
  "notes": "Entreprise intéressante"
}

```


Obtenir une candidature

```
GET /applications/{id}
Authorization: Bearer <token>
```

Mettre à jour une candidature

```
PUT /applications/{id}
Authorization: Bearer <token>
Content-Type: application/json

{
  "status": "interview",
  "notes": "Entretien programmé",
  "interviewDate": "2025-02-01T14:00:00Z"
}
```

Supprimer une candidature

```
DELETE /applications/{id}
Authorization: Bearer <token>
```

Changer le statut

```
PATCH /applications/{id}/status
Authorization: Bearer <token>
Content-Type: application/json

{
  "status": "rejected",
  "rejectionReason": "Compétences non alignées"
}
```



Service des entreprises

Base URL : `http://localhost:3000/companies` **Port direct :** `http://localhost:3003`

Gestion des entreprises

Lister les entreprises

```
GET /companies
Authorization: Bearer <token>
```

Paramètres :

- `search` : Recherche par nom
- `industry` : Filtrer par secteur
- `size` : Filtrer par taille
- `sortBy` : Tri (name, createdAt)

Créer une entreprise

```
POST /companies
Authorization: Bearer <token>
Content-Type: application/json

{
  "name": "Tech Corp",
  "description": "Entreprise technologique innovante",
  "website": "https://techcorp.com",
  "industry": "Technologie",
  "size": "50-200",
  "contactEmail": "contact@techcorp.com",
  "address": "123 Tech Street, Paris"
}
```

Obtenir une entreprise

```
GET /companies/{id}
Authorization: Bearer <token>
```

Mettre à jour une entreprise

```
PUT /companies/{id}
Authorization: Bearer <token>
Content-Type: application/json
```

```
{
  "name": "Tech Corp Updated",
  "description": "Description mise à jour"
}
```

Supprimer une entreprise

```
DELETE /companies/{id}
Authorization: Bearer <token>
```

Contacts d'une entreprise

```
GET /companies/{id}/contacts
Authorization: Bearer <token>
```



Service des contacts

Base URL : `http://localhost:3000/contacts` **Port direct :** `http://localhost:3004`

Gestion des contacts

Lister les contacts

```
GET /contacts
Authorization: Bearer <token>
```

Créer un contact

```
POST /contacts
Authorization: Bearer <token>
Content-Type: application/json

{
  "companyId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Jean Dupont",
  "email": "jean.dupont@techcorp.com",
  "phone": "+33123456789",
```

```
"position": "CTO",  
"linkedin": "https://linkedin.com/in/jeandupont"  
}
```

Obtenir un contact

```
GET /contacts/{id}  
Authorization: Bearer <token>
```

Mettre à jour un contact

```
PUT /contacts/{id}  
Authorization: Bearer <token>  
Content-Type: application/json  
  
{  
  "name": "Jean Dupont",  
  "position": "CEO",  
  "notes": "Contact important"  
}
```

Historique d'un contact

```
GET /contacts/{id}/history  
Authorization: Bearer <token>
```



Service des entretiens

Base URL : `http://localhost:3000/interviews` **Port direct :** `http://localhost:3005`

Gestion des entretiens

Lister les entretiens

```
GET /interviews  
Authorization: Bearer <token>
```

Planifier un entretien

```
POST /interviews
Authorization: Bearer <token>
Content-Type: application/json

{
  "applicationId": "550e8400-e29b-41d4-a716-446655440000",
  "type": "phone",
  "scheduledAt": "2025-02-01T14:00:00Z",
  "duration": 60,
  "location": "Téléphone",
  "interviewer": "Marie Martin",
  "notes": "Premier entretien technique"
}
```

Mettre à jour un entretien

```
PUT /interviews/{id}
Authorization: Bearer <token>
Content-Type: application/json

{
  "status": "completed",
  "notes": "Entretien réussi, très bonne impression",
  "rating": 4
}
```

Ajouter des notes

```
POST /interviews/{id}/notes
Authorization: Bearer <token>
Content-Type: application/json

{
  "content": "Notes détaillées de l'entretien...",
  "isPrivate": false
}
```

Base URL : `http://localhost:3000/calls` **Port direct :** `http://localhost:3006`

Gestion des appels

Lister les appels

```
GET /calls
Authorization: Bearer <token>
```

Enregistrer un appel

```
POST /calls
Authorization: Bearer <token>
Content-Type: application/json

{
  "contactId": "550e8400-e29b-41d4-a716-446655440000",
  "type": "outbound",
  "subject": "Suivi candidature",
  "notes": "Discussion positive sur le poste",
  "duration": 15,
  "outcome": "positive",
  "nextAction": "Envoyer CV mis à jour",
  "scheduledAt": "2025-01-28T10:00:00Z"
}
```

Mettre à jour un appel

```
PUT /calls/{id}
Authorization: Bearer <token>
Content-Type: application/json

{
  "status": "completed",
  "notes": "Appel terminé avec succès"
}
```



Service des événements

Base URL : `http://localhost:3000/events` **Port direct :** `http://localhost:3007`

Gestion des événements

Lister les événements

```
GET /events
Authorization: Bearer <token>
```

Paramètres :

- `startDate` : Date de début
- `endDate` : Date de fin
- `type` : Type d'événement

Créer un événement

```
POST /events
Authorization: Bearer <token>
Content-Type: application/json

{
  "title": "Entretien Tech Corp",
  "description": "Entretien technique",
  "startDate": "2025-02-01T14:00:00Z",
  "endDate": "2025-02-01T15:00:00Z",
  "type": "interview",
  "location": "Bureau Paris",
  "reminders": [15, 60],
  "relatedApplicationId": "550e8400-e29b-41d4-a716-446655440000"
}
```

Événements à venir

```
GET /events/upcoming
Authorization: Bearer <token>
```



Service de suivi

Base URL : `http://localhost:3000/followups` **Port direct :** `http://localhost:3008`

Gestion du suivi

Lister les suivis

```
GET /followups
Authorization: Bearer <token>
```

Créer un suivi

```
POST /followups
Authorization: Bearer <token>
Content-Type: application/json

{
  "applicationId": "550e8400-e29b-41d4-a716-446655440000",
  "type": "email",
  "scheduledAt": "2025-01-30T09:00:00Z",
  "template": "followup_after_interview",
  "customMessage": "Bonjour, suite à notre entretien...",
  "priority": "high"
}
```

Marquer comme terminé

```
PATCH /followups/{id}/complete
Authorization: Bearer <token>
```

Service des profils

Base URL : `http://localhost:3000/profiles` **Port direct :** `http://localhost:3009`

Gestion des profils

Obtenir le profil

```
GET /profiles/me
Authorization: Bearer <token>
```

Mettre à jour le profil

PUT /profiles/me

Authorization: Bearer <token>

Content-Type: application/json

```
{
  "name": "John Doe",
  "bio": "Développeur passionné",
  "skills": ["JavaScript", "React", "Node.js"],
  "experience": "5 ans",
  "location": "Paris",
  "website": "https://johndoe.dev",
  "linkedin": "https://linkedin.com/in/johndoe",
  "github": "https://github.com/johndoe"
}
```

Préférences

PUT /profiles/preferences

Authorization: Bearer <token>

Content-Type: application/json

```
{
  "notifications": {
    "email": true,
    "push": true,
    "reminders": true
  },
  "privacy": {
    "profileVisible": true,
    "showEmail": false
  },
  "dashboard": {
    "defaultView": "kanban",
    "itemsPerPage": 20
  }
}
```

Changer l'avatar

POST /profiles/avatar

Authorization: Bearer <token>

Content-Type: multipart/form-data

```
avatar: <file>
```

Service de notifications

Base URL : `http://localhost:3000/notifications` **Port direct :** `http://localhost:3010`

Gestion des notifications

Lister les notifications

```
GET /notifications
Authorization: Bearer <token>
```

Envoyer une notification

```
POST /notifications
Authorization: Bearer <token>
Content-Type: application/json

{
  "type": "email",
  "recipient": "user@example.com",
  "template": "application_status_changed",
  "data": {
    "applicationId": "550e8400-e29b-41d4-a716-446655440000",
    "oldStatus": "applied",
    "newStatus": "interview"
  }
}
```

Marquer comme lue

```
PATCH /notifications/{id}/read
Authorization: Bearer <token>
```

Paramètres de notification

```
GET /notifications/settings
Authorization: Bearer <token>
```

```
PUT /notifications/settings
Authorization: Bearer <token>
Content-Type: application/json

{
  "emailNotifications": true,
  "pushNotifications": true,
  "reminderNotifications": true,
  "frequency": "immediate"
}
```

Service de workflows

Base URL : `http://localhost:3000/workflows` **Port direct :** `http://localhost:3011`

Gestion des workflows

Lister les workflows

```
GET /workflows
Authorization: Bearer <token>
```

Créer un workflow

```
POST /workflows
Authorization: Bearer <token>
Content-Type: application/json

{
  "name": "Process de recrutement",
  "description": "Workflow standard pour les candidatures",
  "triggers": [
    {
      "event": "application.created",
      "conditions": {
        "company.size": "large"
      }
    }
  ]
}
```

```

    }
  }
],
"steps": [
  {
    "id": "step1",
    "name": "Envoi email confirmation",
    "type": "email",
    "delay": 0,
    "template": "application_confirmation"
  },
  {
    "id": "step2",
    "name": "Relance après 7 jours",
    "type": "email",
    "delay": 604800,
    "conditions": {
      "application.status": "applied"
    }
  }
]
}

```

Exécuter un workflow

```

POST /workflows/{id}/execute
Authorization: Bearer <token>
Content-Type: application/json

{
  "context": {
    "applicationId": "550e8400-e29b-41d4-a716-446655440000",
    "userId": "user-id"
  }
}

```



Service de dashboard

Base URL : `http://localhost:3000/dashboard` **Port direct :** `http://localhost:3007`

Analytics et métriques

Vue d'ensemble

```
GET /dashboard/overview
Authorization: Bearer <token>
```

Réponse :

```
{
  "success": true,
  "data": {
    "stats": {
      "totalApplications": 125,
      "activeApplications": 45,
      "interviewsScheduled": 12,
      "offersReceived": 3,
      "rejections": 65
    },
    "recentActivity": [
      {
        "type": "interview_scheduled",
        "applicationId": "550e8400-e29b-41d4-a716-446655440000",
        "company": "Tech Corp",
        "date": "2025-01-27T14:00:00Z"
      }
    ],
    "trends": {
      "applicationsThisMonth": 25,
      "interviewsThisMonth": 8,
      "responseRate": 0.75
    }
  }
}
```

Analytics détaillés

```
GET /dashboard/analytics
Authorization: Bearer <token>
```

Rapports

```
GET /dashboard/reports
Authorization: Bearer <token>
```

Export de données

POST /dashboard/export

Authorization: Bearer <token>

Content-Type: application/json

```
{
  "format": "pdf",
  "data": ["applications", "companies", "contacts"],
  "filters": {
    "dateRange": {
      "start": "2025-01-01",
      "end": "2025-01-31"
    }
  }
}
```



Service de sécurité

Base URL : http://localhost:3000/security **Port direct :** http://localhost:3017

Audit et sécurité

Logs d'audit

GET /security/audit

Authorization: Bearer <token>

Paramètres :

- **userId** : Filtrer par utilisateur
- **action** : Filtrer par action
- **startDate** : Date de début
- **endDate** : Date de fin

Alertes de sécurité

POST /security/alert

Authorization: Bearer <token>

Content-Type: application/json

```
{
```

```
"type": "suspicious_activity",
"severity": "high",
"description": "Tentatives de connexion multiples",
"metadata": {
  "ip": "192.168.1.100",
  "userAgent": "Mozilla/5.0...",
  "attempts": 5
}
```

Menaces détectées

```
GET /security/threats
Authorization: Bearer <token>
```



Service de métriques système

Base URL : `http://localhost:3000/system-metrics` **Port direct :** `http://localhost:3018`

Métriques système

Métriques temps réel

```
GET /system-metrics
Authorization: Bearer <token>
```

Réponse :

```
{
  "success": true,
  "data": {
    "cpu": {
      "usage": 45.2,
      "load": [1.2, 1.5, 1.8],
      "cores": 8
    },
    "memory": {
      "used": 8.2,
      "total": 16.0,
      "percentage": 51.3
    }
  }
}
```

```
{
  "disk": {
    "used": 120.5,
    "total": 500.0,
    "percentage": 24.1
  },
  "network": {
    "rx": 1024,
    "tx": 2048,
    "connections": 150
  }
},
"timestamp": "2025-01-27T10:00:00Z"
}
```

Historique des métriques

```
GET /system-metrics/history
Authorization: Bearer <token>
```

Configurer une alerte

```
POST /system-metrics/alert
Authorization: Bearer <token>
Content-Type: application/json

{
  "metric": "cpu.usage",
  "threshold": 80,
  "operator": ">",
  "duration": 300,
  "notification": {
    "type": "email",
    "recipients": ["admin@example.com"]
  }
}
```



Service de déploiement

Base URL : `http://localhost:3000/deployment` **Port direct :** `http://localhost:3016`

Gestion des déploiements

Statut des déploiements

```
GET /deployment/status
Authorization: Bearer <token>
```

Lancer un déploiement

```
POST /deployment/deploy
Authorization: Bearer <token>
Content-Type: application/json

{
  "version": "4.0.1",
  "environment": "production",
  "services": ["api-gateway", "auth-service", "application-service"],
  "strategy": "rolling"
}
```

Rollback

```
POST /deployment/rollback
Authorization: Bearer <token>
Content-Type: application/json

{
  "version": "4.0.0",
  "reason": "Bug critique dans l'API"
}
```

Historique des déploiements

```
GET /deployment/history
Authorization: Bearer <token>
```



Service de statistiques Docker

Base URL : `http://localhost:3000/docker-stats` **Port direct :** `http://localhost:3015`

Statistiques Docker

Statistiques temps réel

```
GET /docker-stats
Authorization: Bearer <token>
```

Réponse :

```
{
  "success": true,
  "data": {
    "containers": [
      {
        "id": "550e8400e29b41d4a716446655440000",
        "name": "jobbingtrack-api-gateway",
        "status": "running",
        "cpu": 12.5,
        "memory": 256.8,
        "network": {
          "rx": 1024,
          "tx": 2048
        }
      }
    ],
    "images": {
      "total": 25,
      "size": "2.5GB"
    },
    "volumes": {
      "total": 8,
      "used": "1.2GB"
    }
  }
}
```

Conteneurs actifs

```
GET /docker-stats/containers
Authorization: Bearer <token>
```

Utilisation des ressources

```
GET /docker-stats/resources
Authorization: Bearer <token>
```

Configurer une alerte

```
POST /docker-stats/alert
Authorization: Bearer <token>
Content-Type: application/json

{
  "container": "api-gateway",
  "metric": "memory.usage",
  "threshold": 512,
  "unit": "MB"
}
```

Codes d'erreur

Codes HTTP standard

Code	Description
200	Succès
201	Ressource créée
204	Succès sans contenu
400	Requête invalide
401	Non authentifié
403	Non autorisé
404	Ressource non trouvée
409	Conflit
422	Erreur de validation
429	Trop de requêtes
500	Erreur serveur
502	Bad Gateway

Format des réponses d'erreur

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Les données fournies ne sont pas valides",
    "details": [
      {
        "field": "email",
        "message": "L'email est requis et doit être valide"
      },
      {
        "field": "password",
        "message": "Le mot de passe doit contenir au moins 8 caractères"
      }
    ]
  },
  "timestamp": "2025-01-27T10:00:00Z"
}
```

Codes d'erreur métier

Code	Description
VALIDATION_ERROR	Erreur de validation des données
AUTHENTICATION_ERROR	Échec d'authentification
AUTHORIZATION_ERROR	Permissions insuffisantes
NOT_FOUND	Ressource introuvable
DUPLICATE_ENTRY	Ressource déjà existante
SERVICE_UNAVAILABLE	Service temporairement indisponible
RATE_LIMIT_EXCEEDED	Limite de requêtes dépassée
INVALID_TOKEN	Token invalide ou expiré
QUOTA_EXCEEDED	Limite utilisateur dépassée
EXTERNAL_API_ERROR	Erreur d'API externe

Authentication

Headers d'authentification

Tous les endpoints protégés nécessitent un token JWT :

```
Authorization: Bearer <access_token>
```

Exemple :

```
GET /applications
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI1NTBlODQl
```

Rôles et permissions

Rôles système

- **admin** : Accès complet à tous les endpoints et données
- **manager** : Gestion d'équipe, analytics avancées
- **user** : Accès aux données personnelles et entreprises
- **viewer** : Accès en lecture seule

Permissions granulaires

```
{
  "permissions": [
    "applications:read",
    "applications:write",
    "applications:delete",
    "companies:read",
    "companies:write",
    "contacts:read",
    "contacts:write",
    "interviews:read",
    "interviews:write",
    "calls:read",
    "calls:write",
    "events:read",
    "events:write",
    "notifications:read",
    "notifications:write"
```

```
]
}
```

Refresh Token

Pour maintenir une session active, utilisez le refresh token :

```
POST /auth/refresh
Content-Type: application/json

{
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```



Exemples d'utilisation

Client JavaScript/TypeScript

```
// Configuration API
const API_BASE = 'http://localhost:3000';
const TOKEN_KEY = 'jobbingtrack_token';

class JobbingTrackAPI {
  constructor() {
    this.baseUrl = API_BASE;
  }

  // Requête générique avec authentification
  async request(endpoint, options = {}) {
    const token = localStorage.getItem(TOKEN_KEY);

    const config = {
      headers: {
        'Content-Type': 'application/json',
        ...(token && { Authorization: `Bearer ${token}` }),
        ...options.headers,
      },
      ...options,
    };

    const response = await fetch(`${this.baseUrl}${endpoint}`, config);

    if (!response.ok) {
```

```

    const error = await response.json();
    throw new Error(error.error?.message || `HTTP ${response.status}`);
  }

  return response.json();
}

// Authentification
async login(email, password) {
  const response = await this.request('/auth/login', {
    method: 'POST',
    body: JSON.stringify({ email, password }),
  });

  localStorage.setItem(TOKEN_KEY, response.data.accessToken);
  return response;
}

async logout() {
  await this.request('/auth/logout', { method: 'POST' });
  localStorage.removeItem(TOKEN_KEY);
}

// Candidatures
async getApplications(filters = {}) {
  const query = new URLSearchParams(filters);
  return this.request(`/applications?${query}`);
}

async createApplication(applicationData) {
  return this.request('/applications', {
    method: 'POST',
    body: JSON.stringify(applicationData),
  });
}

// Entreprises
async getCompanies() {
  return this.request('/companies');
}

async createCompany(companyData) {
  return this.request('/companies', {
    method: 'POST',
    body: JSON.stringify(companyData),
  });
}

// Contacts
async getContacts() {
  return this.request('/contacts');
}

```

```

async createContact(contactData) {
  return this.request('/contacts', {
    method: 'POST',
    body: JSON.stringify(contactData),
  });
}

// Dashboard
async getDashboard() {
  return this.request('/dashboard/overview');
}
}

// Utilisation
const api = new JobbingTrackAPI();

// Connexion
await api.login('user@example.com', 'password123');

// Récupérer les candidatures
const applications = await api.getApplications({ status: 'applied' });

// Créer une nouvelle candidature
const newApp = await api.createApplication({
  companyId: '550e8400-e29b-41d4-a716-446655440000',
  position: 'Développeur Full Stack',
  status: 'applied',
  location: 'Paris'
});

```

Client cURL

```

# Variables
API_BASE="http://localhost:3000"
TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

# Connexion
curl -X POST $API_BASE/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "user@example.com",
    "password": "password123"
  }'

# Récupérer les candidatures
curl -X GET $API_BASE/applications \
  -H "Authorization: Bearer $TOKEN"

```



```

# Créer une entreprise
curl -X POST $API_BASE/companies \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "name": "Tech Corp",
    "description": "Entreprise technologique",
    "website": "https://techcorp.com",
    "industry": "Technologie",
    "size": "50-200"
  }'

# Créer un contact
curl -X POST $API_BASE/contacts \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "companyId": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Jean Dupont",
    "email": "jean.dupont@techcorp.com",
    "phone": "+33123456789",
    "position": "CTO"
  }'

# Planifier un entretien
curl -X POST $API_BASE/interviews \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "applicationId": "550e8400-e29b-41d4-a716-446655440000",
    "type": "video",
    "scheduledAt": "2025-02-01T14:00:00Z",
    "duration": 60,
    "location": "Google Meet",
    "interviewer": "Marie Martin",
    "notes": "Entretien technique"
  }'

# Dashboard
curl -X GET $API_BASE/dashboard/overview \
  -H "Authorization: Bearer $TOKEN"

```

Client Python

```

import requests
import json

```

```

class JobbingTrackAPI:
    def __init__(self, base_url='http://localhost:3000'):
        self.base_url = base_url
        self.token = None

    def _request(self, method, endpoint, **kwargs):
        headers = kwargs.get('headers', {})
        if self.token:
            headers['Authorization'] = f'Bearer {self.token}'

        response = requests.request(
            method,
            f'{self.base_url}{endpoint}',
            headers=headers,
            **kwargs
        )

        if not response.ok:
            error = response.json() if response.headers.get('content-type', '').startswith('application/json') else response.text
            raise Exception(f'API Error {response.status_code}: {error}')

        return response.json()

    def login(self, email, password):
        response = self._request(
            'POST',
            '/auth/login',
            json={'email': email, 'password': password}
        )
        self.token = response['data']['accessToken']
        return response

    def get_applications(self, **filters):
        return self._request('GET', '/applications', params=filters)

    def create_application(self, application_data):
        return self._request('POST', '/applications', json=application_data)

    def get_companies(self):
        return self._request('GET', '/companies')

    def create_company(self, company_data):
        return self._request('POST', '/companies', json=company_data)

# Utilisation
api = JobbingTrackAPI()

# Connexion
api.login('user@example.com', 'password123')

# Récupérer les candidatures
applications = api.get_applications(status='applied', limit=10)

```

```
# Créer une entreprise
company = api.create_company({
    'name': 'Tech Corp',
    'description': 'Entreprise technologique innovante',
    'website': 'https://techcorp.com',
    'industry': 'Technologie',
    'size': '50-200'
})
```

Ressources supplémentaires

Documentation	Description	Lien
Architecture	Vue d'ensemble technique	ARCHITECTURE.md
Services	Détail des microservices	SERVICES.md
Développement	Guide de développement	DEVELOPMENT.md
Déploiement	Guide de déploiement	deployment-guide.md
Makefile	Commandes Make	MAKEFILE.md
Tests	Stratégies de test	TESTING-GUIDE.md
Dépannage	Résolution des problèmes	TROUBLESHOOTING.md
Sécurité	Bonnes pratiques	security-guide.md

Navigation

-  [Index Documentation](#)
-  [README Principal](#)
-  [Architecture](#)
-  [Services](#)
-  [Tests](#)
-  [Dépannage](#)

Dernière mise à jour: Octobre 2025 **Version:** 4.0 - API complète avec 15+ services **Endpoints documentés:** 100+ **Services couverts:** 15 microservices + API Gateway **Format:** RESTful JSON API avec authentification JWT